03

Part 3 · Joins, Aggregations & Skew

The expensive operations, how Spark executes them, and how to fix imbalance.

- Join strategies
- Broadcast join
- Join order
- Aggregation
- Data skew
- AQE skew join
- Salting
- UDFs vs built-ins

3.1 Spark Join Strategies

★★★

IN ONE LINE

Broadcast hash, sort-merge and shuffled hash joins differ in whether and how data is shuffled.

A **join strategy** is the physical algorithm Spark uses to match rows. The choice changes runtime by orders of magnitude, and it is the most-asked Spark optimisation topic.

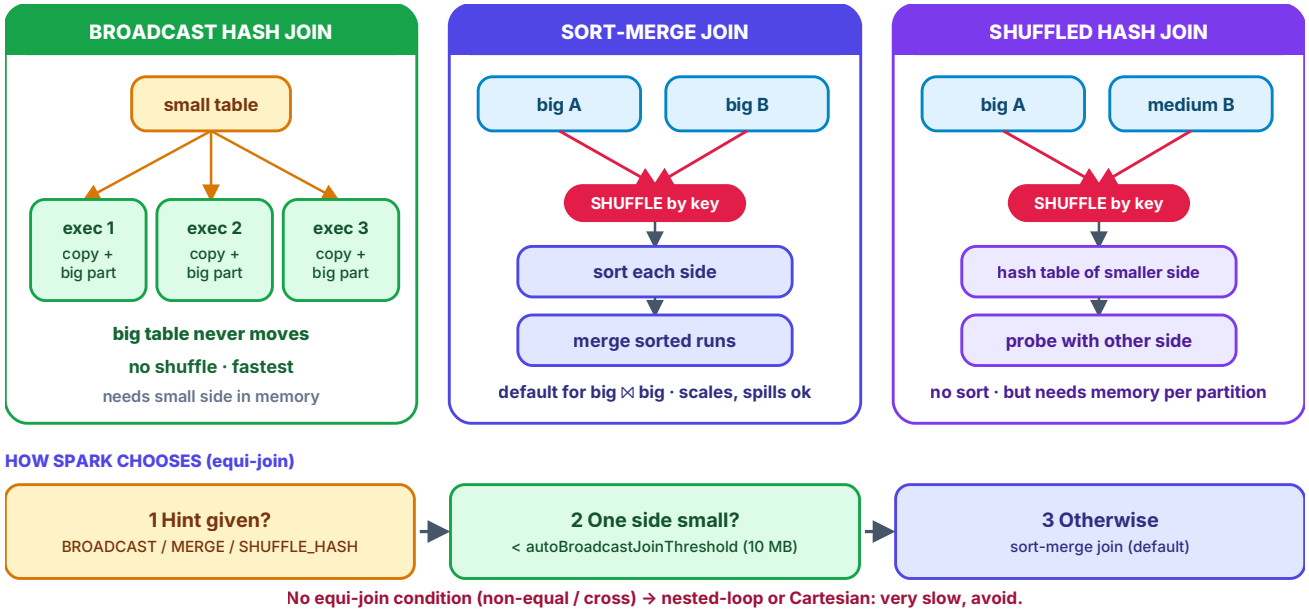


Figure 3.1 — The three main join strategies and how Spark picks one

Strategy	Shuffle?	Best when	Watch out for
Broadcast hash join	No	One side small (default < 10 MB, can be raised or hinted)	Broadcasting something too big → driver/executor OOM
Sort-merge join	Both sides	Both sides large (the default)	Shuffle and sort cost; skewed keys
Shuffled hash join	Both sides	One side clearly smaller (but too big to broadcast)	Needs the build side to fit in memory per partition
Broadcast nested loop / Cartesian	Varies	Non-equality or no join condition	Explosive cost: usually a bug

JOIN HINTS

```
from pyspark.sql.functions import broadcast

fast = orders.join(broadcast(customers), 'customer_id')            # force broadcast
hint = orders.join(customers.hint('shuffle_hash'), 'customer_id')  # ask for shuffled hash
merge = orders.join(returns.hint('merge'), 'order_id')             # force sort-merge
```

INTERVIEW ANGLE

Always answer with **"check with `explain()` which join was chosen"**. Look for `BroadcastHashJoin` vs `SortMergeJoin`. With AQE, Spark can switch a sort-merge join to broadcast at runtime when the real data turns out small (6.3).

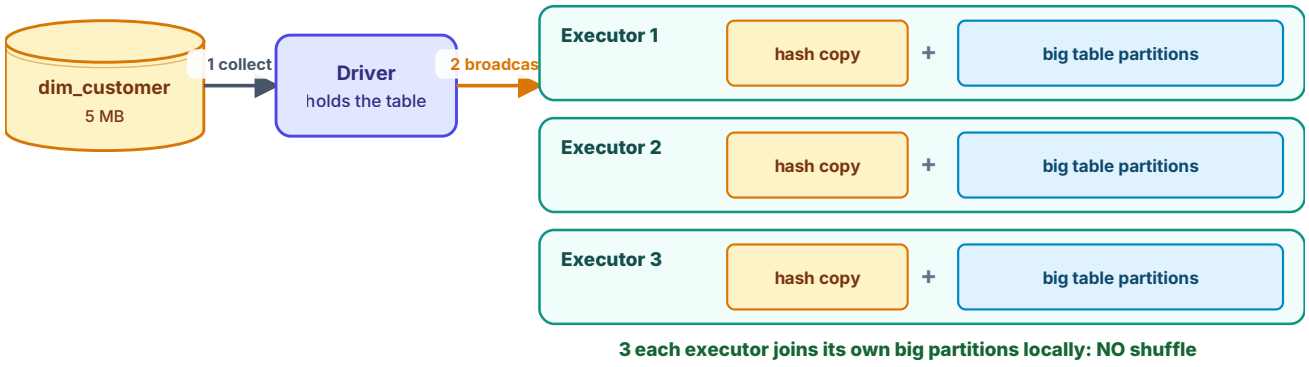
3.2 Broadcast Joins

★★★

IN ONE LINE

Copy the small table to every executor so the large table never has to shuffle.

In a **broadcast join**, Spark ships a full copy of the small table to every executor, then joins locally with each partition of the large table. The big side is **never shuffled**, so this is usually the biggest single join speed-up.



SIZE LADDER



the small side is held by the driver AND every executor: memory cost = size × (executors + 1)

Figure 3.2 — Collect, broadcast, join locally

THREE WAYS TO BROADCAST

```
from pyspark.sql.functions import broadcast

# explicit: guaranteed regardless of size estimates
result = facts.join(broadcast(dim), 'dim_id')

# or raise the automatic threshold (bytes); -1 disables
spark.conf.set('spark.sql.autoBroadcastJoinThreshold', 100 * 1024 * 1024)

# SQL hint
spark.sql('SELECT /*+ BROADCAST(d) */ * FROM facts f JOIN dim d ON f.dim_id = d.id')
```

Use it when

- One side is small after filtering (dimension tables, lookups).
- Join is an **equi-join**.
- You can afford the memory on driver and executors.

Limits & traps

- Hard limit about **8 GB** / 512 M rows; broadcast timeout (default 300 s).
- You cannot broadcast the **preserved side** of an outer join (e.g. the left table in a LEFT JOIN).
- Statistics may be missing, so Spark under-estimates size: use the hint.
- Broadcasting a big table → driver OOM.

INTERVIEW ANGLE

Say: “broadcast the small dimension table to avoid shuffling the big fact table” and explain the trade-off: **memory for network**.

3.3 Join Order and Data Reduction

★★★

IN ONE LINE

Shrink data with filters, projections and pre-aggregation before the expensive join.

The cheapest row to join is the one you never joined. **Reduce data before expensive operations**: filter rows, select only the needed columns, pre-aggregate — so the shuffle moves far fewer bytes.

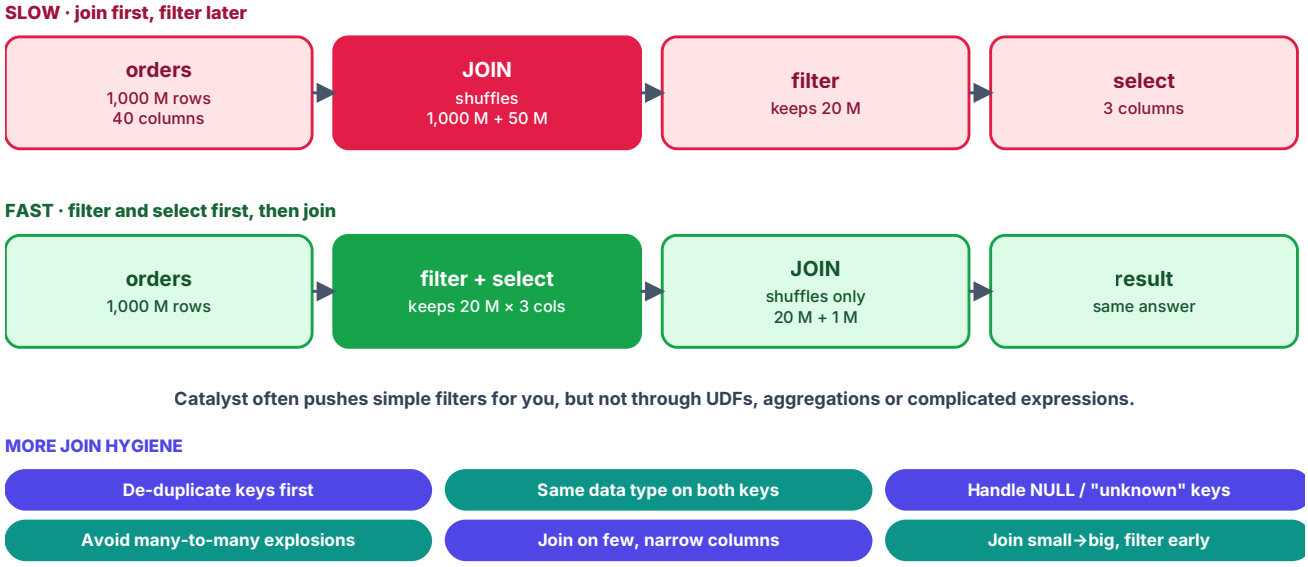


Figure 3.3 — Same result, far less data shuffled

Practice	Effect
Filter early	Fewer rows enter the shuffle.
Select only needed columns	Narrower rows: less to serialise and send.
Pre-aggregate before joining to a summary	Join 1 M summary rows instead of 1 B detail rows.
Keep keys the same type	Int vs string keys can force casts, defeat pushdown, and join nothing.
Watch duplicate keys	Many-to-many joins multiply rows: row count after join ≫ before is a red flag.
NULL keys	NULL never equals NULL in a join, yet all NULLs hash to one partition (skew).

INTERVIEW ANGLE

Check the **SQL tab** in the Spark UI: it shows rows output by each operator. If rows explode after a join, you have duplicate keys; if a huge scan feeds a join that then drops most rows, filter earlier.

3.4 Aggregation Optimization

★★★

IN ONE LINE

Partial aggregation cuts shuffle volume; distinct-style aggregates and collect_* are the expensive ones.

Aggregations (`groupBy`, `distinct`, `agg`, windows) are wide transformations. Spark softens the cost with **partial (map-side) aggregation**: each task first collapses its own rows, so far less data is shuffled. Anything that defeats partial aggregation is expensive.

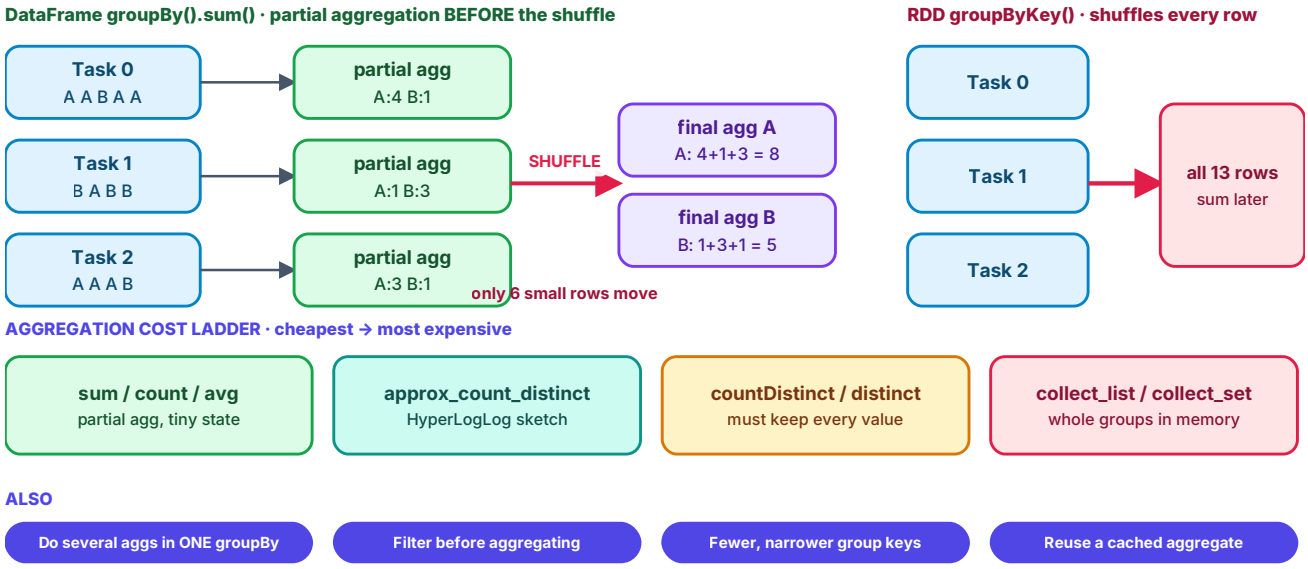


Figure 3.4 — Partial aggregation shrinks the shuffle; some aggregates cannot be partial

Pattern	Cost	Better
`groupBy().agg(sum, count, avg)`	Low: partial aggregation	Put all aggregates in **one** `agg()`, not several groupBys
`countDistinct(col)`	High	`approx_count_distinct` if ~2% error is acceptable
`df.distinct()` on wide rows	High: shuffles every column	`dropDuplicates([key_cols])` or distinct on a subset
`collect_list / collect_set`	Very high memory	Aggregate numerically, or limit group size
Window `partitionBy(high-skew key)`	Shuffle + sort + skew	Pre-filter, or compute with a groupBy + join
RDD `groupByKey`	Shuffles all values	`reduceByKey` / DataFrame `groupBy().agg`

CHEAP AGGREGATION

```
from pyspark.sql import functions as F

# one pass, many aggregates, approximate distinct
summary = (orders.groupBy('city')
    .agg(F.sum('amount').alias('revenue'),
         F.avg('amount').alias('avg_order'),
         F.approx_count_distinct('customer_id').alias('customers')))
```

3.5 Data Skew

★★★

IN ONE LINE

Uneven key distribution creates a few huge partitions and stragglers that stall a stage.

Data skew means keys are unevenly distributed, so a few partitions hold most of the data. Those tasks run far longer than the rest and the whole stage waits for them. It is the #1 cause of the "99 of 100 tasks finished, one task runs forever" symptom.

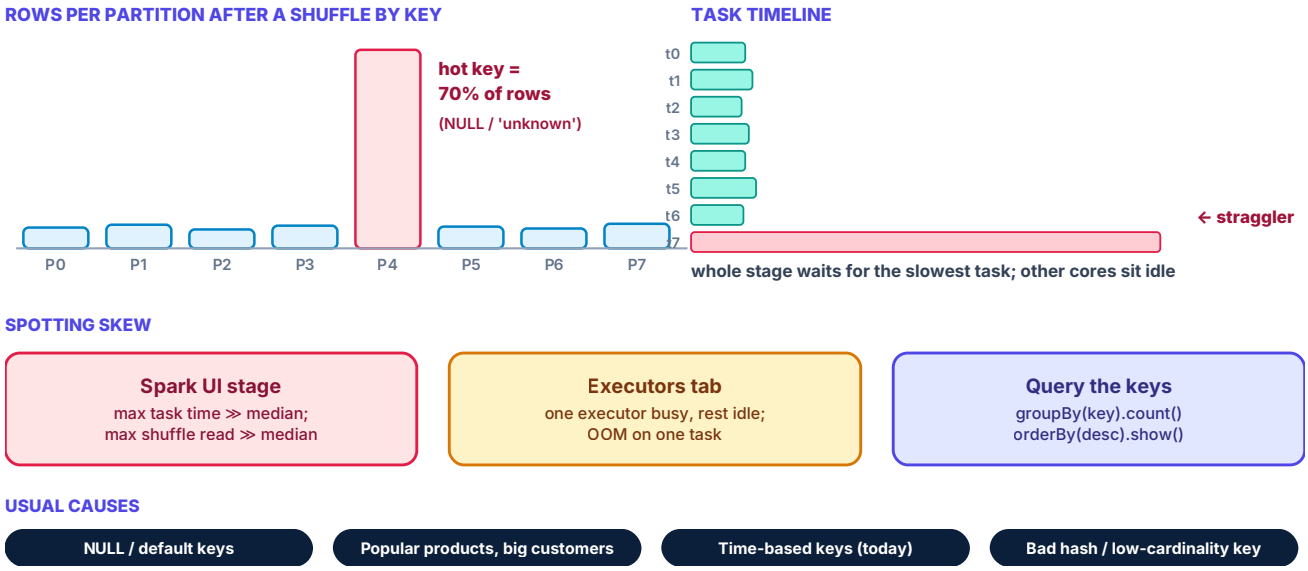


Figure 3.5 — One hot partition, one straggler task

Symptom	Typical result
One task takes minutes while the rest take seconds	Stage time = slowest task
Max shuffle read per task » median	Memory spill or OOM on that task
Job fails with executor OOM only on some keys	Hot key does not fit in memory
Adding more cores does not help	Extra cores sit idle waiting

DIAGNOSE SKEW

```
# find the hot keys
(df.groupBy('customer_id').count()
   .orderBy(F.desc('count')).show(10))

# how bad? ratio of max to median partition
from pyspark.sql.functions import spark_partition_id
sizes = df.repartition('customer_id').groupBy(spark_partition_id()).count()
```

REMEMBER

Fixes, from simplest to most manual: **filter junk keys** → **broadcast** the small side → **AQE skew join** (3.6) → **salting** (3.7) → **isolate the hot key** and process it separately.

3.6 Skew Handling with AQE

★★★

IN ONE LINE

AQE detects oversized join partitions at runtime and splits them automatically.

With **Adaptive Query Execution (AQE)**, Spark looks at the **actual shuffle partition sizes at runtime**. If a partition in a sort-merge (or shuffled hash) join is far bigger than the median, it automatically **splits** it into smaller sub-partitions and replicates the matching data from the other side. No code change needed.

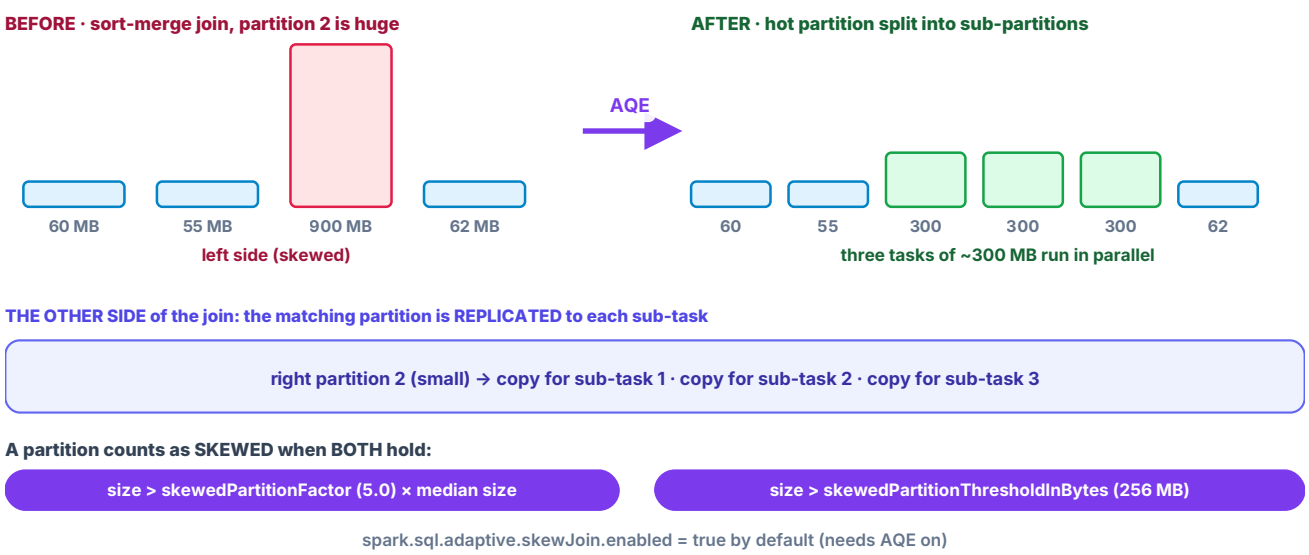


Figure 3.6 — AQE splits a skewed partition and replicates the matching side

Setting	Default	Meaning
`spark.sql.adaptive.enabled`	true	Master switch for AQE
`spark.sql.adaptive.skewJoin.enabled`	true	Handle skew in joins
`…skewJoin.skewedPartitionFactor`	5.0	Skewed if > factor × median partition size
`…skewJoin.skewedPartitionThresholdInBytes`	256 MB	And also larger than this absolute size
`…advisoryPartitionSizeInBytes`	64 MB	Target size of the pieces

It helps

- Skewed **joins** (sort-merge, shuffled hash).
- Unknown hot keys (no code change).

It does NOT help

- Skewed **aggregations / window** on one huge group.
- Skew so extreme that one key is bigger than a task can hold.
- Cases where the skewed side cannot be split (e.g. some outer-join sides).

INTERVIEW ANGLE

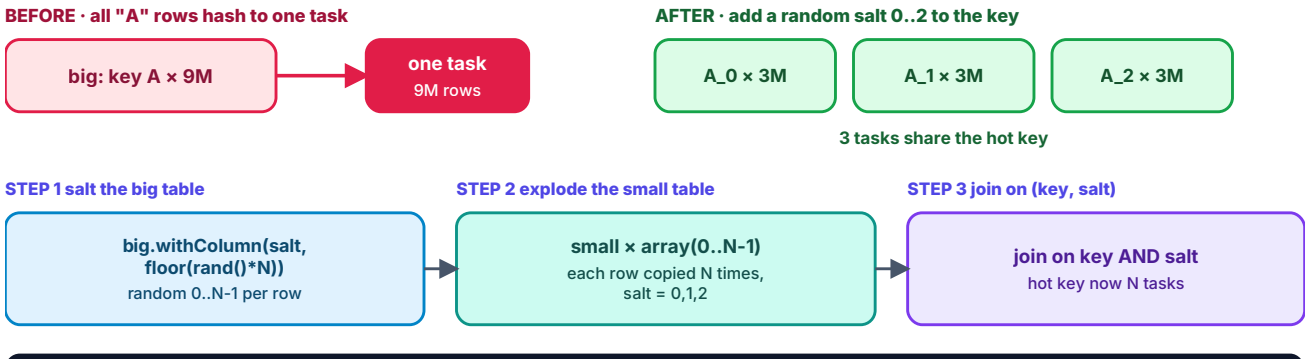
Order of answers: *"First check AQE is on and tune thresholds (lower factor / threshold if skew is mild); if it is not enough, salt the key."*

3.7 Salting

★★☆

IN ONE LINE
Spread a hot key across N partitions using a random suffix, then recombine.

Salting is a manual trick for skew: add a random suffix ("salt") to the hot key so its rows are spread across many partitions, then combine the results. Use it when AQE and broadcast are not enough.




```
N = 8 # number of salt buckets
big = big.withColumn('salt', (F.rand() * N).cast('int'))
small = small.withColumn('salt', F.explode(F.array(*[F.lit(i) for i in range(N)])))
out = big.join(small, ['key', 'salt']).drop('salt')
```

Figure 3.7 — Spread one hot key over N salted keys

Salting a join

Salt the **big** side randomly (0..N-1). **Replicate** every row of the small side N times, once per salt. Join on `(key, salt)`. Cost: the small side grows N×.

Salting an aggregation

Two-stage: first `groupBy(key, salt)` to get partial results, then `groupBy(key)` to combine. Works for sum, count, min, max.

SKEWED GROUPBY

```
# two-stage salted aggregation
salted = df.withColumn('salt', (F.rand() * 16).cast('int'))
partial = salted.groupBy('key', 'salt').agg(F.sum('amt').alias('s'))
final = partial.groupBy('key').agg(F.sum('s').alias('total'))
```

Choose	When
Filter / drop junk keys	NULLs and placeholders carry no business value
Broadcast	One side is small
AQE skew join	Mild/moderate skew in a join
Salting	Severe skew, aggregation skew, or AQE is unavailable
Isolate hot keys	Handle 1-2 known giants separately, union results

COMMON TRAP
Salting increases code complexity and the size of one side. Pick N sensibly (usually 4–32). Over-salting just creates overhead; under-salting leaves a straggler.

3.8 Avoid Python UDFs: Built-ins and Pandas UDFs

★★★

IN ONE LINE

Built-in functions are optimised in the JVM; Python UDFs are slow opaque black boxes.

A **UDF (user-defined function)** lets you run your own Python on each row. Convenient, but for Catalyst it is an opaque **black box**, and Python UDFs move every row between the JVM and a Python process. Built-in functions avoid all of that.

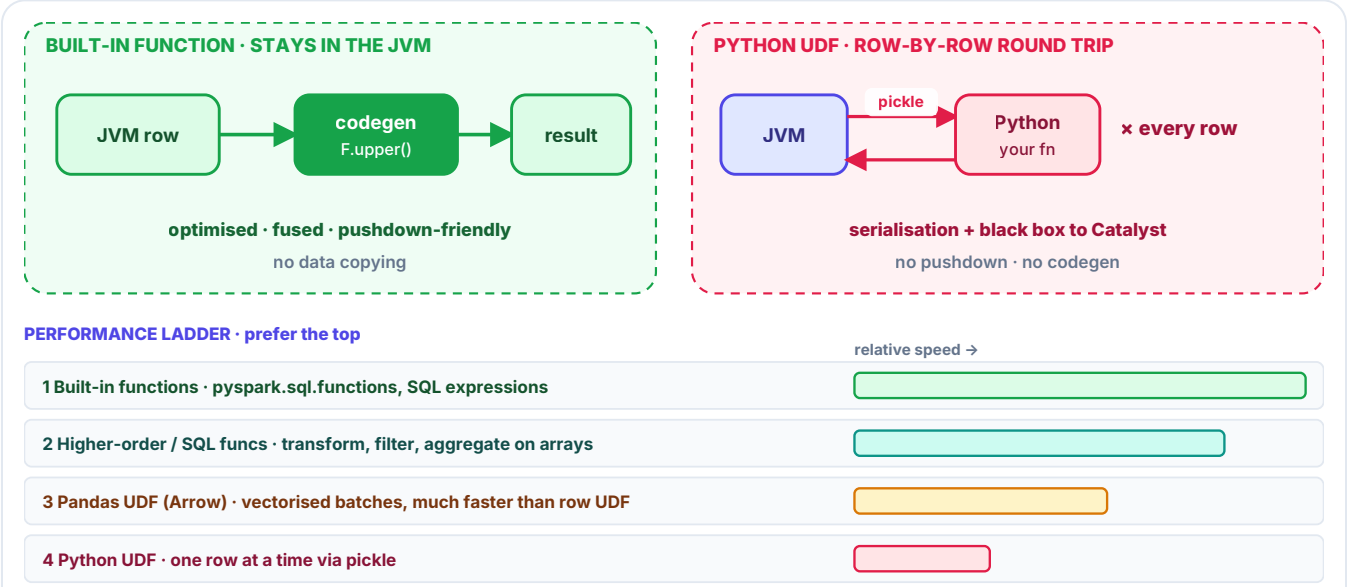


Figure 3.8 — Built-ins stay in the JVM; Python UDFs pay a serialisation tax

SAME IDEA, FOUR SPEEDS

```python
from pyspark.sql import functions as F
import pandas as pd
from pyspark.sql.functions import pandas_udf, udf

# 1 best: built-in
df.withColumn('clean', F.upper(F.trim('name')))

# 3 acceptable: vectorised pandas UDF (Arrow)
@pandas_udf('double')
def tax(amount: pd.Series) -> pd.Series:
    return amount * 0.18

# 4 last resort: plain Python UDF
@udf('string')
def slow_fn(x): return x[::-1]
```

Why UDFs hurt

- Per-row (de)serialisation between JVM and Python.
- No predicate pushdown through them.
- No whole-stage code generation.
- Harder to optimise and to test at scale.

Before writing a UDF, try

- `when/otherwise`, `regexp_extract`, `split`
- Array functions: `transform`, `filter`, `aggregate`
- Spark SQL expressions via `F.expr()`
- Joins against a lookup table

INTERVIEW ANGLE

Strong answer: *"I rewrite UDFs with built-ins; if impossible, I use a **pandas UDF** so work is vectorised through Arrow."* Add that **Scala/Java UDFs** avoid Python serialisation but are still opaque to Catalyst.